# OCaml の C は圏論の C!

Haochen Kotoi-Xie
haochenx / github.com/haochenx

俺の勉強会 #8
30OCT 2026

# 前回の LT のタイトル

OCaml の ml は
**Meta Language の ML!**

証明を操作するための言語、ML。

今日は、**もう一つの由来**の話。

# Caml = CAM + L

**Categorical** Abstract Machine Language

CAM：圏論に由来する抽象機械

関数を実行するための、仮想的な計算機。

初期の Caml は、この CAM にコンパイルした。

**Categorical = 圏論的な**

出典：Inria, FAQ（フランス語版）, caml.inria.fr/resources/doc/faq/general.fr.html

# 前回の発表のあと、こんなやり取りが




やり取りの要旨

**中野先生（Keisuke Nakano）／ 8 月 26 日**

Caml の語源は CAML。
ml は Meta Language の ML ではない。

**自分（haochenx）／ 8 月 31 日**

CAM+L と ML、両方では？
調べてみたら……

**フランス語の FAQ にあった。**

# 公式 FAQ には「もう一つの系譜」

*La deuxième filiation de ce nom est ML*

「この名前のもう一つの系譜は **ML**」

（発表者訳）

**CAM + L**

圏論に由来する抽象機械
そのための言語

**ML**

Meta Language
言語としての系譜

**名前に、両方が掛かっている。**

出典：Inria, FAQ（フランス語版）, caml.inria.fr/resources/doc/faq/general.fr.html

OCaml 公式の歴史ページも、CAM と ML を掛けた名前と説明：ocaml.org/history

# 圏論の入口：関数を「つなぐ」

身近な例：集合を「対象」、関数を「射」として見る。

$$A \xrightarrow{f} B \xrightarrow{g} C$$

合成： $(g \circ f)(x) = g(f(x))$

**OCaml で書くと**

```
let compose g f x = g (f x)
```

**つなぐときのルール**

括り方を変えても同じ（結合律）。

入力をそのまま返す関数もある（恒等射）。

**計算の「つながり方」に、数学の言葉を与える。**

ここでは、必ず値を返す純粋な関数で直感を説明。OCaml の副作用や非停止を含む意味論全体とは区別する。

# 直積閉圏（CCC）と指数対象

Cartesian Closed Category：終対象と、任意の 2 対象の直積・指数対象を持つ圏。

**終対象** $1$ — `unit`

**直積** $A \times B$ — `'a * 'b`

**指数対象** $B^A$ (exponential) — `'a -> 'b` 関数型

**関数適用** $\mathrm{eval} : B^A \times A \longrightarrow B$

**カリー化**：次の射が一対一に対応

$$f : X \times A \longrightarrow B \quad \longleftrightarrow \quad \mathrm{curry}(f) : X \longrightarrow B^A$$

$\lambda$ 式で関数を作ることと、適用することの数学。

型の表記は OCaml 風。対応の対象は純粋な単純型付き $\lambda$ 計算。
出典：A. M. Pitts, *Category Theory*, Lectures 5–8（Cambridge），講義資料

# その数学が、処理系の設計につながった

関数の生成と適用を、圏の演算として捉える。

**直積閉圏（CCC）** と **$\lambda$ 計算**

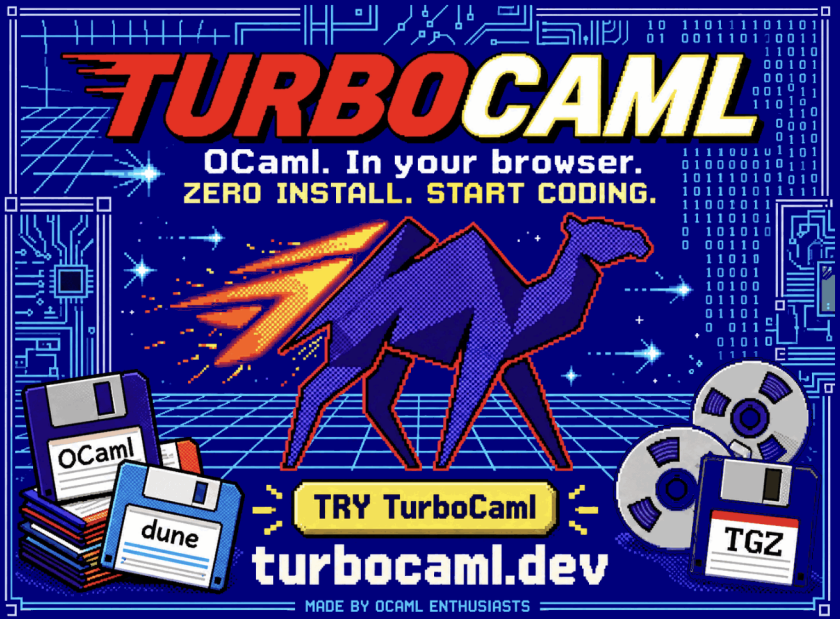
この対応に基づく計算の表現を、実行する仕組みに。

**CAM** $\longrightarrow$ Caml $\longrightarrow$ OCaml

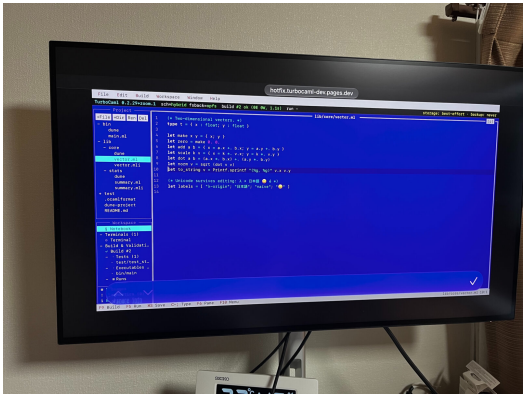
名前の C に、**圏論由来の歴史**が残っている。

G. Cousineau, P.-L. Curien, M. Mauny, *The categorical abstract machine* (1987).
doi:10.1016/0167-6423(87)90020-7

歴史上の由来の話。Caml Light 以降は CAM を使用していない：ocaml.org/history


TURBOCAML
OCaml. In your browser.
ZERO INSTALL. START CODING.
OCaml
dune
TGZ
TRY TurboCaml
turbocaml.dev
MADE BY OCAML ENTHUSIASTS

# iPhone でも、OCaml を書ける




写真：発表者による iPhone での利用例

## iPhone でも ちゃんと使える。

ブラウザで開いて、
書く。実行する。

型チェック・補完も。

日本語表示にも対応。

# 今日から OCaml、始めてみませんか？

## turbocaml.dev

開くだけで、OCaml を書き始められる。

Notebook で、まずはこの 2 行。

```
let square x = x * x;;
square 8;;
```

結果は **64**。型も自動で付く。

圏論が気になった人も、まずは OCaml から！



ここからアクセス